

EDS Lab Assignments

Practical 3

Name: Dhanashri Mamidwar

PRN: 202501040059

3.1.1 Numpy Array Operations

Problem Statement:

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim
- The shape of the array using shape
- The total number of elements in the array using size

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Code:

```
import numpy as np
rows, cols = map(int, input().split())
elements = []

for _ in range (rows):
    elements.extend(map(int, input().split()))

array = np.array(elements).reshape(rows, cols)
print(array)
print(array.ndim)
print(array.shape)
print(array.size)
```


3.2.1 Numpy: Matrix Operations

Problem Statement:

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** ($\text{matrix_a} + \text{matrix_b}$)
2. **Subtraction** ($\text{matrix_a} - \text{matrix_b}$)
3. **Element-wise Multiplication** ($\text{matrix_a} * \text{matrix_b}$)
4. **Matrix Multiplication** ($\text{matrix_a} \cdot \text{matrix_b}$)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
addition_result = matrix_a + matrix_b
print("Addition (A + B):")
```

```
print(addition_result)
```

```
# Subtraction
```

```
subtraction_result = matrix_a - matrix_b
```

```
print("Subtraction (A - B):")
```

```
print(subtraction_result)
```

```
# Multiplication (element-wise)
```

```
elementwise_multiplication_result = matrix_a * matrix_b
```

```
print("Element-wise Multiplication (A * B):")
```

```
print(elementwise_multiplication_result)
```

```
# Matrix multiplication (dot product)
```

```
matrix_multiplication_result = np.dot(matrix_a, matrix_b)
```

```
print("A dot B:")
```

```
print(matrix_multiplication_result)
```

```
# Transpose
```

```
transpose_a = matrix_a.T
```

```
print("Transpose of A:")
```

```
print(transpose_a)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a sidebar shows the project structure with a file named '3.2.1. Numpy: Matrix Operations'. The main editor area contains a Python script that performs various matrix operations using NumPy. The script includes comments for each operation: Addition, Subtraction, Element-wise Multiplication, Matrix Multiplication (dot product), and Transpose of Matrix A. The script takes user input for two 3x3 matrices, matrix_a and matrix_b, and prints the results of each operation. The output window on the right shows the execution results, which match the expected output from the script. The interface also includes a terminal, test cases, and a debug console.

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Addition
11 addition_result = matrix_a + matrix_b
12 print("Addition (A + B):")
13 print(addition_result)
14
15 # Subtraction
16 subtraction_result = matrix_a - matrix_b
17 print("Subtraction (A - B):")
18 print(subtraction_result)
19
20 # Multiplication (element-wise)
21 elementwise_multiplication_result = matrix_a * matrix_b
22 print("Element-wise Multiplication (A * B):")
23 print(elementwise_multiplication_result)
24
25 # Matrix multiplication (dot product)
26 multiplication_result = np.dot(matrix_a, matrix_b)
27 print("A dot B:")
28 print(multiplication_result)
29
30 # Transpose
31 transpose_a = matrix_a.T
32 print("Transpose of A:")
33 print(transpose_a)
```

CODETANTRA

HomeLearn Anywhere

202501040959@mttce.ac.inSupportLogout

12.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:
You are required to compute and display the results of the following matrix operations:
1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a * matrix_b`)
5. Transpose of Matrix A

Input Format:
• The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
• Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:
The program should display the results of the operations in the following order:
1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

Average time0.009 s9.25 msMaximum time0.010 s10.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Enter matrix A:

2 3
4 5
7 8

Enter matrix B:

3 2
5 3
6 4

Addition (A + B):

[[2 3 4]
[5 6 7]
[9 9 10]]

Subtraction (A - B):

[[0 1]
[3 4]
[6 7]]

Element-wise Multiplication (A * B):

[[13 7]
[15 15]
[24 24 24]]

Transpose of A:

[[1 4 7]
[2 5 8]
[3 6 9]]

Actual output

Enter matrix A:

2 3
4 5
7 8

Enter matrix B:

3 2
5 3
6 4

Addition (A + B):

[[2 3 4]
[5 6 7]
[9 9 10]]

Subtraction (A - B):

[[0 1]
[3 4]
[6 7]]

Element-wise Multiplication (A * B):

[[13 7]
[15 15]
[24 24 24]]

Transpose of A:

[[1 4 7]
[2 5 8]
[3 6 9]]

Test case 2

CODETANTRA

HomeLearn Anywhere

202501040959@mttce.ac.inSupportLogout

12.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:
You are required to compute and display the results of the following matrix operations:
1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a * matrix_b`)
5. Transpose of Matrix A

Input Format:
• The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
• Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:
The program should display the results of the operations in the following order:
1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

Average time0.009 s9.25 msMaximum time0.010 s10.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Enter matrix A:

2 3
4 5
7 8

Enter matrix B:

10 12 13
14 15 16
17 18 19

Addition (A + B):

[[12 16 21]
[21 20 21]
[24 26 28]]

Subtraction (A - B):

[[-8 -9 -5]
[-5 -9 -11]
[-10 -10 -10]]

Element-wise Multiplication (A * B):

[[20 40 184]
[28 36 88]
[110 144 171]]

Transpose of A:

[[1 4 7]
[2 5 8]
[3 6 9]]

Actual output

Enter matrix A:

2 3
4 5
7 8

Enter matrix B:

10 12 13
14 15 16
17 18 19

Addition (A + B):

[[12 16 21]
[21 20 21]
[24 26 28]]

Subtraction (A - B):

[[-8 -9 -5]
[-5 -9 -11]
[-10 -10 -10]]

Element-wise Multiplication (A * B):

[[20 40 184]
[28 36 88]
[110 144 171]]

Transpose of A:

[[1 4 7]
[2 5 8]
[3 6 9]]

Test case 2

3.2.2 Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement:

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
b = np.vstack((arr1, arr2))
print("Vertical Stack:")
print(b)
```

Output:

CODETANTRAHomeLearn Anywhere

202501040059@mitace.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal Stacking (hstack)
11 a=np.hstack((arr1,arr2))
12 print("Horizontal Stack:")
13 print(a)
14
15 # Perform vertical stacking (vstack)
16 b=np.vstack((arr1,arr2))
17 print("Vertical Stack:")
18 print(b)
19
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501040059@mitace.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

Average time: 0.010 sMaximum time: 0.011 s

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Enter Array1:

Enter Array1:

Enter Array2:

Enter Array2:

Horizontal Stack:

Horizontal Stack:

Vertical Stack:

Vertical Stack:

Test case 2

CODETANTRAHomeLearn Anywhere

202501040059@mitace.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

Average time: 0.010 sMaximum time: 0.011 s

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Enter Array1:

Enter Array1:

Enter Array2:

Enter Array2:

Horizontal Stack:

Horizontal Stack:

Vertical Stack:

Vertical Stack:

Test case 2

3.2.3 Numpy: Custom Sequence Generation

Problem Statement:

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code:

```
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()
a = np.arange(start, stop, step)
# Print the generated sequence
print(a)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and input/output formats are visible. The main editor shows the Python code for generating a sequence using `np.arange()`. The output pane at the bottom shows the execution results, including the generated sequence `[2 5 7 9]` for the first test case and `[18]` for the second test case. The interface also shows a terminal, test cases, and a debugger.

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 a = np.arange(start, stop, step)
10
11 # Print the generated sequence
12 print(a)
```

Average time: 0.007 s
Maximum time: 0.010 s
6.75 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1

Expected output

```
[2]
[5]
[7]
[9]
```

Actual output

```
[2 5 7 9]
```

Test case 2

Expected output

```
[18]
```

Actual output

```
[18]
```

Terminal Test cases

Next

3.2.4 Numpy: Arithmetic and Statistical Operations, Mathematical Operations and Bitwise Operators

Problem Statement:

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```
import numpy as np
```

```
def array_operations(A, B):
    # Convert A and B to NumPy arrays
    A = np.array(A)
    B = np.array(B)

    # Arithmetic Operations
    sum_result = A + B
    diff_result = A - B
    prod_result = A * B

    # Statistical Operations
    mean_A = np.mean(A)
```

```
median_A = np.median(A)
std_dev_A = np.std(A)
```

Bitwise Operations

```
and_result = A & B
```

```
or_result = A | B
```

```
xor_result = A ^ B
```

Output results with one space between each element

```
print("Element-wise Sum:", ' '.join(map(str, sum_result)))
print("Element-wise Difference:", ' '.join(map(str, diff_result)))
print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}")
print(f"Median of A: {median_A}")
print(f"Standard Deviation of A: {std_dev_A}")
print("Bitwise AND:", ' '.join(map(str, and_result)))
print("Bitwise OR:", ' '.join(map(str, or_result)))
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

A = list(map(int, input().split())) # Elements of array A

B = list(map(int, input().split())) # Elements of array B

array_operations(A, B)

Output:

```
12.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operations
You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:
1. Arithmetic Operations:
  • Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
  • Calculate the mean, median, and standard deviation of array A.
3. Bitwise Operations:
  • Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A0, OR B0).
Input Format:
  • The first line contains space-separated integers representing the elements of array A.
  • The second line contains space-separated integers representing the elements of array B.
Output Format:
  • For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.
Sample Test Cases
```

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to Numpy arrays
6     A = np.array(A)
7     B = np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A + B
11    diff_result = A - B
12    prod_result = A * B
13
14    # Statistical Operations
15    mean_A = np.mean(A)
16    median_A = np.median(A)
17    std_dev_A = np.std(A)
18
19    # Bitwise Operations
20    and_result = A & B
21    or_result = A | B
22    xor_result = A ^ B
23
24    # Output results with one space between each element
25    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
26    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
27    print("Element-wise Product:", ' '.join(map(str, prod_result)))
28
29    print(f"Mean of A: {mean_A}")
30    print(f"Median of A: {median_A}")
31    print(f"Standard Deviation of A: {std_dev_A}")
32
33    print("Bitwise AND:", ' '.join(map(str, and_result)))
34    print("Bitwise OR:", ' '.join(map(str, or_result)))
35    print("Bitwise XOR:", ' '.join(map(str, xor_result)))
36
37 A = list(map(int, input().split())) # Elements of array A
38 B = list(map(int, input().split())) # Elements of array B
39 array_operations(A, B)
```


3.2.5 Numpy: Copying and Viewing Arrays

Problem Statement:

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
- After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>

Code:

```
import numpy as np
inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
```

```
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Output:

The screenshot shows a coding challenge on the CodeTANTRA platform. The challenge is titled "3.2.6. Numpy: Copying and Viewing Arrays". The instructions state: "The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by: • Creating a view of the original_array and assigning it to view_array. • Creating a copy of the original_array and assigning it to copy_array. After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not."

Input Format:
• A single line of space-separated integers.

Output Format:
• After modifying the view:
Original array after modifying view: original_array
View array: view_array
• After modifying the copy:
Original array after modifying copy: original_array
Copy array: copy_array

Sample Test Cases

The code editor shows the following Python code:

```
1 import numpy as np
2
3 input_list = list(map(int, input().split()))
4
5 # Original array
6 original_array = np.array(input_list)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 88
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21 print("Original array after modifying copy:", original_array)
22 print("Copy array:", copy_array)
23
```

The terminal output shows the results of the code execution for two test cases:

Test case 1

Expected output: 88 20 30 40 50 60 70 80

Actual output: 88 20 30 40 50 60 70 80

Original array after modifying view: [88 20 30 40 50 60 70 80]
View array: [88 20 30 40 50 60 70 80]
Original array after modifying copy: [88 20 30 40 50 60 70 80]
Copy array: [88 88 30 40 50 60 70 80]

Test case 2

Expected output: 88 3 3 4 5

Actual output: 88 3 3 4 5

Original array after modifying view: [88 3 3 4 5]
View array: [88 3 3 4 5]
Original array after modifying copy: [88 3 3 4 5]
Copy array: [88 88 3 4 5]

3.2.6 Searching, Sorting, Counting, Broadcasting

Problem Statement:

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where search_value appears in array1 and print these indices.
- 2. Counting:** Count how many times count_value appears in array1 and print the count.
- 3. Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- 4. Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Code:

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))
```

```
# Find indices where value matches in array1
a = np.where(array1 == search_value)[0]
print(a)
```

```
# Count occurrences in array1
b = np.count_nonzero(array1 == count_value)
print(b)
```

```
# Broadcasting addition
c = array1 + broadcast_value
print(c)
```

```
# Sort the first array
d = np.sort(array1)
print(d)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a panel titled '3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting' provides instructions for the task. It states that the code takes a single array, `array1`, as input from the user. The instructions are as follows:

- 1. Searching:** Find the indices where `search_value` appears in `array1` and print these indices.
- 2. Counting:** Count how many times `count_value` appears in `array1` and print the count.
- 3. Broadcasting:** Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
- 4. Sorting:** Sort `array1` in ascending order and print the sorted array.

The **Input Format:** section specifies four inputs: a single line of space-separated integers for `array1`, an integer for `search_value`, an integer for `count_value`, and an integer for `broadcast_value`.

The **Output Format:** section specifies four outputs: the indices where `search_value` occurs, the count of occurrences of `count_value`, the array after adding `broadcast_value` to each element, and the sorted array.

On the right, the code editor shows the implementation of these tasks:

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 a = np.where(array1 == search_value)[0]
13 print(a)
14
15 # Count occurrences in array1
16 b = np.count_nonzero(array1 == count_value)
17 print(b)
18
19 # Broadcasting addition
20 c = array1 + broadcast_value
21 print(c)
22
23 # Sort the first array
24 d = np.sort(array1)
25 print(d)
```

At the bottom, there are buttons for 'Run', 'Clear', 'Submit', and 'Next'.

CODETANTRA

HomeLearn Anywhere

20251043256@mit2025.ac.inSupportLogout

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
- Counting**: Count how many times `count_value` appears in `array1` and print the count.
- Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
- Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing `array1`.
- An integer `search_value` represents the value to search for in the array.
- An integer `count_value` represents the value to count in the array.
- An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

- The indices where `search_value` occurs in `array1`.
- The count of occurrences of `count_value` in `array1`.
- The array after adding the `broadcast_value` to each element.
- The sorted array.

Sample Test Cases

arrayOpt...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
```

Average time0.006 s6.75 msMaximum time0.007 s7.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

[1 3 3 4 4 4]

[1 1 1 2 2 2]

Test case 2

Expected output

Actual output

Value to search: 4

Value to count: 3

Value to add: 5

[]

0

[-7 -0 -9 -10 -11]

[1 2 3 4 5]

Terminal

Test cases

Prev

Next

Submit

3.2.7 Student Data Analysis and Operations

Problem Statement:

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.

- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Code:

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

roll = a[:, 0].astype(float)
s1 = a[:, 1]
s2 = a[:, 2]
s3 = a[:, 3]

# 1. Print all student details
print("All student Details:\n", a)

# 2. print total students
print("Total Students:", len(a))

# 3. Print all student Roll numbers
print("All Student Roll Nos", roll)

# 4. Print subject 1 marks
print("Subject 1 Marks", s1)

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(s2))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(s3))

# 7. Print All subject marks
print("All subject marks:", a[:, 1:])

# 8. print Total marks of students
total = s1 + s2 + s3
print("Total Marks", total)

# 9. print average marks of each student
```

```

print(np.round(total/3, 1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis = 0))

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean([s1, s2], axis = 1))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean([s1, s3], axis = 1))

# 13. print Roll number who got maximum marks in Subject 3
print("Roll no who got maximum marks in Subject 3", float(roll[np.argmax(s3)]))

# 14. print Roll number who got minimum marks in Subject 2
print("Roll no who got minimum marks in Subject 2", float(roll[np.argmin(s2)]))

# 15. print Roll number who got 24 marks in Subject 2
print("Roll no who got 24 marks in Subject 2", roll[s2 == 24].astype(float).reshape(1, -1))

# 16. print count of students who got marks in Subject 1 < 40
print("Count of students who got marks in Subject 1 < 40", np.sum(s1 < 40))

# 17. print count of students who got marks in Subject 2 > 90
print("Count of students who got marks in Subject 2 > 90:", np.sum(s2 > 90))

# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90,
axis = 0))

# 19. print count of subjects in which each student got marks >= 90
print("Roll no:", roll)
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis =
1))

# 20. Print S1 marks in ascending order
print(np.sort(s1))

# 21. Print S1 marks >= 50 and <= 90
print(a[(s1 >= 50) & (s1 <= 90)].astype(float))
print(a.astype(float))

# 22. Print the index position of marks 79
print(np.where(s1 == 79))

```

Output:

CODETANTRA

Home Learn Anywhere

202501040259@mitac.ac.in Support Logout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details. Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students. Determine the total number of students in the dataset.
- Print all student roll numbers. Extract and print the roll numbers of all students.
- Print Subject 1 marks. Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2. Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3. Identify the highest marks in Subject 3.
- Print all subject marks. Display the marks of all students for each subject.
- Find total marks of students. Compute the total marks for each student across all subjects.
- Find the average marks of each student. Compute the average marks for each student.
- Find average marks of each subject. Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2. Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3. Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3. Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2. Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2. Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1. Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2. Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >=90 in each subject. Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >=90. Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order. Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1. Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1. Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Operations

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=";", skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n", a )
7
8 # 2. print total students
9 print("Total Students:",a.shape[0] )
10
11 # 3. Print all student Roll numbers
12 print("All Student Roll Nos",a[:,0] ----)
13
14 # 4. Print subject 1 marks
15 print("Subject 1 Marks",a[:,1] )
16
17 # 5. print minimum marks of Subject 2
18 print("Min marks in Subject 2",np.min(a[:,2]))-----)
19
20 # 6. print maximum marks of Subject 3
21 print("Max marks in Subject 3",np.max(a[:,3]) )
22
23 # 7. Print All subject marks
24 print("All subject marks:", a[:,1:] ----)
25
26 # 8. Print Total marks of students
27 print("Total Marks", np.sum(a[:,1:],axis=1) -----)
28
29 # 9. print average marks of each student
30 print(np.round(np.mean(a[:,1:],axis=1),1))
31
32 # 10. print average marks of each subject
33 print("Average Marks of each subject", np.round(np.mean(a[:,1:],axis=0),1))
34
35 # 11. print average marks of S1 and S2
36 print("Average Marks of S1 and S2", np.round(np.mean(a[:,1:2],axis=0),1))
37
38 # 12. print average marks of S1 and S3
39 print("Average Marks of S1 and S3", np.round(np.mean(a[:, [1,3]],axis=0),1))
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
```

Terminal Test cases

CODETANTRA

Home Learn Anywhere

202501040259@mitac.ac.in Support Logout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details. Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students. Determine the total number of students in the dataset.
- Print all student roll numbers. Extract and print the roll numbers of all students.
- Print Subject 1 marks. Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2. Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3. Identify the highest marks in Subject 3.
- Print all subject marks. Display the marks of all students for each subject.
- Find total marks of students. Compute the total marks for each student across all subjects.
- Find the average marks of each student. Compute the average marks for each student.
- Find average marks of each subject. Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2. Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3. Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 2. Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2. Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2. Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1. Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2. Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >=90 in each subject. Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >=90. Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order. Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1. Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1. Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Operations

```
38 # 12. print average marks of S1 and S3
39 print("Average Marks of S1 and S3", np.round(np.mean(a[:, [1,3]],axis=0),1))
40
41 # 13. print Roll number who got maximum marks in Subject 3
42 print("Roll no who got maximum marks in Subject 3", a[np.argmax(a[:,3],0)])
43
44 # 14. print Roll number who got minimum marks in Subject 2
45 print("Roll no who got minimum marks in Subject 2", a[np.argmin(a[:,2],0)])
46
47 # 15. print Roll number who got 24 marks in Subject 2
48 print("Roll no who got 24 marks in Subject 2", a[a[:,2] == 24,0].reshape(-1,1))
49
50 # 16. print count of students who got marks in Subject 1 < 40
51 print("Count of students who got marks in Subject 1 < 40", np.sum(a[:,1] < 40))
52
53 # 17. print count of students who got marks in Subject 2 > 90
54 print("Count of students who got marks in Subject 2 > 90", np.sum(a[:,2] > 90))
55
56 # 18. print count of students in each subject who got marks >= 90
57 print("Count of students in each subject who got marks >= 90:", np.sum(a[:,1:] >= 90, axis=0))
58
59 # 19. print count of subjects in which each student got marks >= 90
60 print("Count of subjects in which student got marks >= 90:", np.sum(a[:,1:] >= 90, axis=1))
61
62 # 20. Print S1 marks in ascending order
63 print(np.sort(a[:, 1]))
64
65 # 21. Print S1 marks >= 50 and <= 90
66 print(a[a[:,1] >= 50] & (a[:,1] <= 90))
67 print(a)
68
69 # 22. Print the index position of marks 79
70 print(np.where(a[:,1] == 79))
71
72
73
74
75
76
```

Terminal Test cases

CODETANTRAHomeLearn Anywhere

20251040059@mitace.ac.inSupportLogout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations.

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Execution time: 0.013 s13.00 msMaximum time: 0.013 s13.00 ms1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

All student details:
[[1901, 47, 77, 88],
[1902, 78, 88, 77],
[1903, 45, 54, 89],
[1904, 88, 98, 45],
[1905, 78, 88, 99],
[1906, 68, 78, 34],
[1907, 79, 32, 45],
[1908, 46, 45, 77],
[1909, 89, 32, 88],
[1910, 79, 24, 33]]
Total students: 10
All student roll nos: [1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910]
Subject 1 marks: [47, 78, 45, 88, 78, 68, 79, 46, 89, 79]
Min marks in Subject 2: 32
Max marks in Subject 3: 99
All subject marks: [[47, 77, 88],
[78, 88, 77],
[45, 54, 89],
[88, 98, 45],
[78, 88, 99],
[68, 78, 34],
[79, 32, 45],
[46, 45, 77],
[89, 32, 88],
[79, 24, 33]]
Total marks: [232, 243, 190, 231, 265, 182, 136, 168, 289, 134]
[77.3 81, 43.3 77, 88.3 66.7 45.3 56, 45.7 45.3]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]
Average marks of 11 and 12: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]

All student details:
[[1901, 47, 77, 88],
[1902, 78, 88, 77],
[1903, 45, 54, 89],
[1904, 88, 98, 45],
[1905, 78, 88, 99],
[1906, 68, 78, 34],
[1907, 79, 32, 45],
[1908, 46, 45, 77],
[1909, 89, 32, 88],
[1910, 79, 24, 33]]
Total students: 10
All student roll nos: [1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910]
Subject 1 marks: [47, 78, 45, 88, 78, 68, 79, 46, 89, 79]
Min marks in Subject 2: 32
Max marks in Subject 3: 99
All subject marks: [[47, 77, 88],
[78, 88, 77],
[45, 54, 89],
[88, 98, 45],
[78, 88, 99],
[68, 78, 34],
[79, 32, 45],
[46, 45, 77],
[89, 32, 88],
[79, 24, 33]]
Total marks: [232, 243, 190, 231, 265, 182, 136, 168, 289, 134]
[77.3 81, 43.3 77, 88.3 66.7 45.3 56, 45.7 45.3]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]

PrintResetSubmitNext

CODETANTRAHomeLearn Anywhere

20251040059@mitace.ac.inSupportLogout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations.

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 2: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Execution time: 0.013 s13.00 msMaximum time: 0.013 s13.00 ms1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

All student details:
[[1901, 47, 77, 88],
[1902, 78, 88, 77],
[1903, 45, 54, 89],
[1904, 88, 98, 45],
[1905, 78, 88, 99],
[1906, 68, 78, 34],
[1907, 79, 32, 45],
[1908, 46, 45, 77],
[1909, 89, 32, 88],
[1910, 79, 24, 33]]
Total students: 10
All student roll nos: [1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910]
Subject 1 marks: [47, 78, 45, 88, 78, 68, 79, 46, 89, 79]
Min marks in Subject 2: 32
Max marks in Subject 3: 99
All subject marks: [[47, 77, 88],
[78, 88, 77],
[45, 54, 89],
[88, 98, 45],
[78, 88, 99],
[68, 78, 34],
[79, 32, 45],
[46, 45, 77],
[89, 32, 88],
[79, 24, 33]]
Total marks: [232, 243, 190, 231, 265, 182, 136, 168, 289, 134]
[77.3 81, 43.3 77, 88.3 66.7 45.3 56, 45.7 45.3]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]
Average marks of 11 and 12: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]

All student details:
[[1901, 47, 77, 88],
[1902, 78, 88, 77],
[1903, 45, 54, 89],
[1904, 88, 98, 45],
[1905, 78, 88, 99],
[1906, 68, 78, 34],
[1907, 79, 32, 45],
[1908, 46, 45, 77],
[1909, 89, 32, 88],
[1910, 79, 24, 33]]
Total students: 10
All student roll nos: [1901, 1902, 1903, 1904, 1905, 1906, 1907, 1908, 1909, 1910]
Subject 1 marks: [47, 78, 45, 88, 78, 68, 79, 46, 89, 79]
Min marks in Subject 2: 32
Max marks in Subject 3: 99
All subject marks: [[47, 77, 88],
[78, 88, 77],
[45, 54, 89],
[88, 98, 45],
[78, 88, 99],
[68, 78, 34],
[79, 32, 45],
[46, 45, 77],
[89, 32, 88],
[79, 24, 33]]
Total marks: [232, 243, 190, 231, 265, 182, 136, 168, 289, 134]
[77.3 81, 43.3 77, 88.3 66.7 45.3 56, 45.7 45.3]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]
Average marks of each subject: [71.7 59.8 47.7]
Average marks of 11 and 12: [71.7 59.8]

PrintResetSubmitNext